Software Testing Final Project ——AutoTestDesign

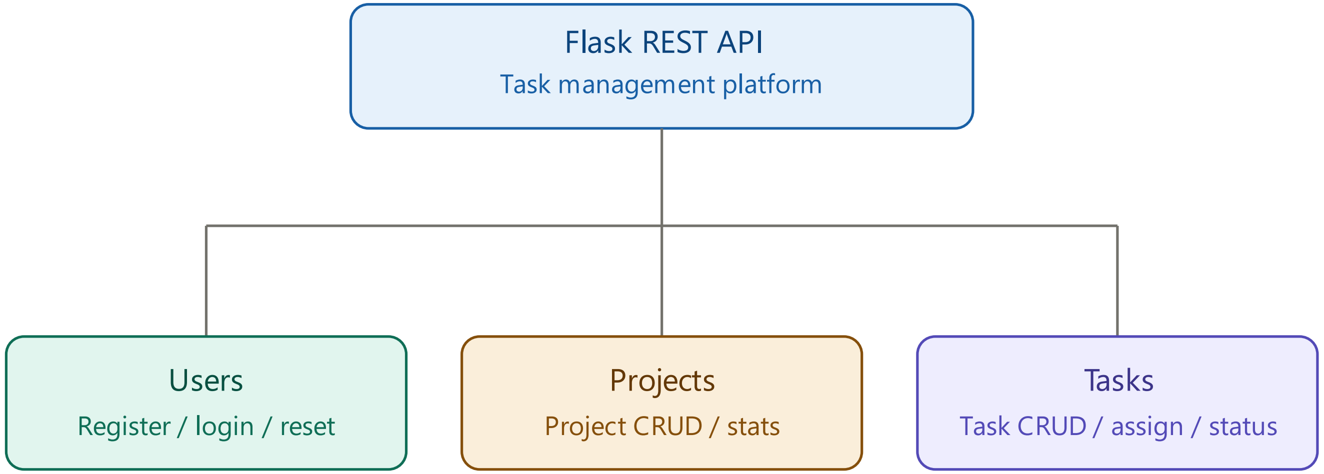
Group6

2351269黄一和 2352744关镜文 2351271黄弋涵   2352048夏浩博

Course Requirements & Implementation

ID	Function Category	Implementation
FR1/1.1	Requirement input & structuring	Extract fields, conditions, and expected behaviors from CSV / text / direct input
FR 2.0	Risk analysis & priority ranking	Generate a risk score, risk level, and test priority for each requirement
FR 3.0	Black-box test design	EP, BVA, and decision tables — three techniques to generate test cases
FR 4.0/5.0	White-box & oracle	Path coverage, state transition, expected-result generation
FR 6.0	Output & export	Generate in a structured format; export Pytest / JSON
FR 7.0	Test suite optimization	Risk-first prioritization, deduplication; designers can review and revise coverage items, strategies, and test cases

Test Target—— Task Management Plantform



Why it is suitable for validating the tool

- It has both field validation and CRUD, assignment, statistics, and status behaviors; in addition, this project manually ported in some bugs, so it can measure whether the tool truly detects requirement deviations.

Workflow

- **1. Requirement Input**
- 2. Risk Analysis
- 3. Black-box Test Design
- 4. White-box Test Design
- 5. Oracle Generation
- 6. Suite Optimization
- 7. Export

Input:

Code directory / CSV requirement file / TXT text / text

Output:

Standardized requirements, fields, conditions, expected behaviors, requirement-code mapping

Workflow

1. Requirement Input
2. **Risk Analysis**
3. Black-box Test Design
4. White-box Test Design
5. Oracle Generation
6. Suite Optimization
7. Export

Users can review, revise, and change risk levels and analyses

Workflow

1. Requirement Input
2. Risk Analysis
3. **Black-box Test Design**
4. White-box Test Design
5. Oracle Generation
6. Suite Optimization
7. Export

Select test requirements

Select test technique:EP BVA DT

Based on requirement fields, input constraints, and condition combinations, the system generates analysis tables and corresponding test cases

Manual review and modification

Workflow

1. Requirement Input
2. Risk Analysis
3. Black-box Test Design
4. White-box Test Design
5. Oracle Generation
6. Suite Optimization
7. Export

白盒测试设计

路径覆盖 & 状态转换 — LLM 分析代码生成测试用例

标准需求

REQ-001: 用户注册

测试技术

PathCoverage × StateTransition ×

生成测试用例

共 12 用例

路径覆盖 状态转换

覆盖率

关联行	覆盖行	覆盖率
107	107	100.0%

覆盖映射

› TC-PC-001 用户注册成功

› TC-PC-002 用户名已存在

ID	从	到	触发	守卫	动作
T1	S1	S2	提交注册请求	请求体为JSON格式	解析请求数据
T2	S2	S3	验证用户名	用户名有效	继续验证密码
T3	S2	S8	验证用户名	用户名无效	返回错误信息
T4	S3	S4	验证密码	密码有效	继续验证邮箱
T5	S3	S8	验证密码	密码无效	返回错误信息
T6	S4	S5	验证邮箱	邮箱有效	继续验证年龄
T7	S4	S8	验证邮箱	邮箱无效	返回错误信息
T8	S5	S6	验证年龄	年龄有效	检查唯一性
T9	S5	S8	验证年龄	年龄无效	返回错误信息
T10	S6	S7	检查唯一性	用户名和邮箱唯一	创建用户

All-States 路径 (6)

- S1 → S2 → S3 → S4 → S5 → S6 → S7 | 完整的成功注册路径
- S1 → S2 → S8 | 用户名验证失败路径
- S1 → S2 → S3 → S8 | 密码验证失败路径
- S1 → S2 → S3 → S4 → S8 | 邮箱验证失败路径
- S1 → S2 → S3 → S4 → S5 → S8 | 年龄验证失败路径
- S1 → S2 → S3 → S4 → S5 → S6 → S8 | 唯一性检查失败路径

Select test requirements

Select test technique: PathCoverage State-Transition

The system generates: covered paths, coverage results, and the state transition table

Manual review and modification

Workflow

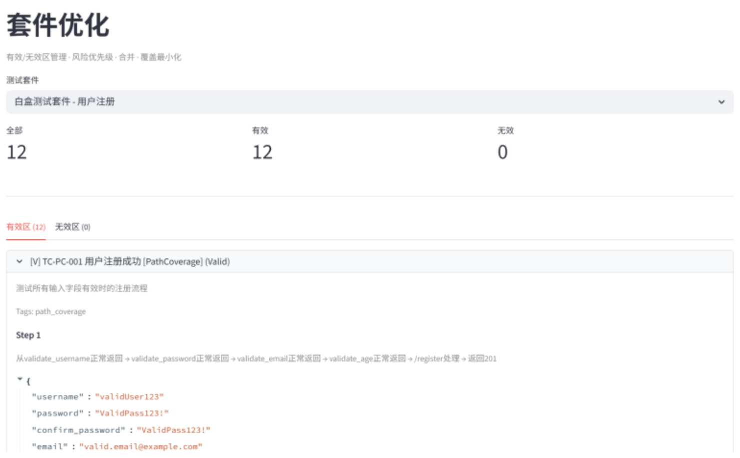
1. Requirement Input
2. Risk Analysis
3. Black-box Test Design
4. White-box Test Design
5. **Oracle Generation**
6. Suite Optimization
7. Export

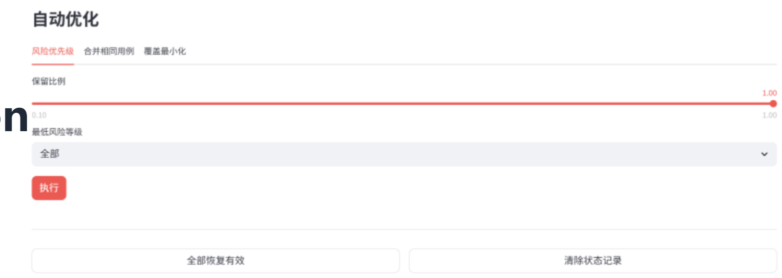
The system generates expected results from the input test data

Generated test cases can be saved to a custom suite

Workflow

1. Requirement Input
2. Risk Analysis
3. Black-box Test Design
4. White-box Test Design
5. Oracle Generation
6. **Suite Optimization**
7. Export





Test items can be manually enabled, disabled, and modified

The system can sort and filter test cases by risk priority, coverage relationships, and duplication

Workflow

1. Requirement Input
2. Risk Analysis
3. Black-box Test Design
4. White-box Test Design
5. Oracle Generation
6. Suite Optimization
7. **Export**

Test cases in the suite can be exported as JSON or as runnable Pytest

Video Demo

Risk Analysis

需求	风险分数	风险	优先级
REQ-001	88	High	High
REQ-002	92	High	High
REQ-003	85	High	High
REQ-004	70	Medium	Medium
REQ-005	65	Medium	Medium
REQ-006	75	Medium	Medium
REQ-007	50	Low	Low

REQ-001~003

High — authentication & account security

REQ-004~006

Medium — core project & task business

REQ-007

Low — read-only statistics logic

Test Design

Test Strategy

Adopt a risk-driven strategy:

High-risk requirements prioritize authentication security;

Medium-risk requirements focus on business rules and key paths;

Low-risk requirements focus on statistics logic and interface consistency.

Technique Combination

Black-box: EP / BVA / DT

White-box: path coverage / state transition

Test Plan

Select REQ-005, REQ-006, REQ-007

Compare three script types: auto-generated, manually verified, and hand-written

Experiment Results

Dimension	`test_generated.py`	`test_generated_refined.py`	`test_human.py`
Source	Auto-generated	Auto-generated, then manually verified	Hand-written from scratch
Test functions	44	32	27
Script's own errors	17	0	0
Real defects found	0	3 (BUG-7, BUG-8, BUG-9)	1 (BUG-9)
Fix / writing time	~3-5 min, auto-generated	~20 min verifying & fixing	~4 hrs of manual design & writing
LLM cost	Input 79,260 / Output 10,828, cost ≈ ¥2	Input 79,260 / Output 10,828, cost ≈ ¥2	¥0

Conclusion:

The main value of the automated tool is more thorough test perspectives, but it produces some of its own bugs.

Hand-written scripts are inherently reliable, but their test perspectives are not rich enough.

Comparison Experiment

Defect	Meaning	Found this round
BUG-7	Assignment/update for an unregistered owner not handled	Found by `test_generated_refined.py`
BUG-8	Completed tasks can still be deleted	Found by `test_generated_refined.py`
BUG-9	A past `due_date` is not rejected	Found by `test_generated_refined.py` and `test_human.py`

As can be seen, automated scripts after manual verification clearly outperform hand-written scripts

Anomaly 1: REQ-005 coverage is only 43.8%

Original result:

associated lines 73, covered lines 32, coverage 43.8%

Investigation found:

The requirement side focused on create, update, and delete, while downplaying the list and detail paths

The requirement-code map pulled in a broad range of task-routing code

Coverage is counted by line, while the uncovered-line hints only show fully uncovered segments

After updating the requirement description:

covered lines rose to 73 lines, coverage became 100%

Anomaly 2: In black-box testing, EP/BVA did not generate test cases

REQ-007 definition:

input_fields is an empty array;

conditions is an empty array;

expected_behaviors has only one: "exists -> returns statistics result"

EP: No explicit set of input fields, and no field-level valid / invalid classes

BVA: No numeric range, length range, or explicit boundary

Not suitable for case generation

The current tool shows some technical adaptability to different requirement types:

when a requirement lacks field-boundary information, the tool does not force out low-quality EP/BVA cases, but is more likely to produce results on decision tables and state transitions

Performance & Generalization

Model	Configuration	Total time	Valid cases	EP	BVA	DT	Verdict
gpt-4o	yunwu.ai / reasoning param disabled	24.414s	22	7.615s / 12 cases	11.060s / 6 cases	5.739s / 4 cases	Most stable this round
deepseek-v4-flash	thinking disabled	108.415s	3	43.897s / 0 cases	60.405s / 0 cases	4.113s / 3 cases	DT works, EP/BVA unstable
gemini-3.5-flash	free API	70.940s	14	16.245s / 8 cases	18.400s / 6 cases	36.295s / 0 cases	EP/BVA works, DT unstable

In terms of generation speed and completeness, under the current configuration, choosing gpt-4o is more appropriate.

Performance & Generalization

This system supports custom projects and generates test cases directly from requirements and the project repository

Project Summary

AutoTestDesign establishes a traceable design chain from requirements to test cases, code paths, and execution scripts

01

Clear advantages in the test-design stage

It can quickly produce black-box analysis tables, white-box coverage ideas, and large-scale candidate test scripts

02

Auto-generate a first draft, with humans doing the key verification

This approach strikes a better balance between time cost and defect-detection capability.。

03

Auto-generated scripts cannot completely skip the manual step

The raw auto-exported scripts had many problems, but after one modest round of manual verification, script quality improved markedly and defect-detection ability was significantly enhanced.

小组分工

2351269黄一和

Responsible for building the overall code structure of the project; implemented requirement input, CSV / text parsing, and standardized requirement modeling; organized the mapping between requirements and code

2352744关镜文

Responsible for risk analysis, black-box and white-box test design; designed and tuned LLM Prompt; implemented EP, BVA, decision tables, path coverage, state transition, and other generation flows

2351271黄弋涵

Responsible for the target Flask application and test-script execution; maintained the Pytest export and run workflow; ran comparison experiments across hand-written, automated, and verified scripts; analyzed defect detection and anomalies

2352048夏浩博

Responsible for Streamlit interface optimization and workflow presentation; organized screenshot materials; unified document format and wording; created the PPT

感谢观看
恳请老师批评指正